

Laporan Praktikum Pemograman Web

Jobsheet 10 dan 11



Oleh:

Muhammad Ramdhani Ariyadin (254107060018)

SIB 2-E

Link Git Hub :

<https://github.com/MuhammadRamdhaniAriyadin/Pemrograman-Web>

JOBSHET 10

1. Kontrol Akses Berbasis Role (Role-Based Access Control / RBAC)

Perbaiki Kode (anggota/hapus.php):

menambahkan pengecekan hak akses setelah require __DIR__ .

'../includes/auth.php';. Jika role pengguna bukan admin, kita set pesan flash error dan mengarahkannya kembali ke list.php

```
// Pengecekan Hak Akses: Hanya 'admin' yang boleh menghapus anggota
if (($SESSION['role'] ?? '') !== 'admin') {
    $_SESSION['flash'] = [
        'type' => 'error',
        'pesan' => 'Akses ditolak! Hanya Admin yang diperbolehkan menghapus anggota.'
    ];
    header('Location: list.php');
    exit;
}
```

Implementasi Kontrol Akses Berbasis Role (RBAC): Telah ditambahkan otorisasi berbasis hak akses pada file anggota/hapus.php dengan memanfaatkan data peran/role pengguna yang tersimpan di dalam \$_SESSION['role']. Sistem memeriksa kualifikasi peran pengguna tepat setelah autentikasi auth.php dipanggil. Jika pengguna berkategori 'petugas' melakukan tindakan penghapusan data anggota, sistem secara otomatis menolak permintaan, membatalkan perintah DELETE, dan memberikan notifikasi error berupa flash message berupa arahan kembali ke halaman list.php. Fitur penghapusan anggota kini terbatas secara eksklusif untuk peran 'admin'

Uji coba

- Login dengan akun yang memiliki role = 'petugas'.
- Buka halaman Daftar Anggota (anggota/list.php).
- Tekan tombol Hapus pada salah satu anggota.



No. Anggota	Nama	Alamat	No. HP	Aksi
2134	ariya	SDGF	13254564542	<button>Edit</button> <button>Hapus</button>

- Data TIDAK terhapus. Pengguna diarahkan kembali ke list.php dengan pesan error: "Akses ditolak! Hanya Admin yang diperbolehkan menghapus anggota."

2. Fitur "Ingat Saya" (Remember Me) & Pembahasan Risiko Keamanan

- Membuat perintah sql untuk mendukung token Remember Me:

```
ALTER TABLE users ADD COLUMN remember_token VARCHAR(255) NULL;
```

- Menambahkan checkbox

```
<p>
  <label>
    <input type="checkbox" name="remember" value="1"> Ingat Saya
  </label>
</p>
```

- Modifikasi alur login

```
// Jika pengguna mencentang "Ingat Saya"
if ($remember) {
    $token = bin2hex(random_bytes(32));
    $tokenHash = hash('sha256', $token);

    // Simpan hash token di database
    $updateStmt = $pdo->prepare("UPDATE users SET remember_token = :token WHERE id = :id");
    $updateStmt->execute(['token' => $tokenHash, 'id' => $user['id']]);

    // Simpan cookie selama 30 hari
    $cookieValue = $user['id'] . ':' . $token;
    setcookie('remember_me', $cookieValue, [
        'expires' => time() + (86400 * 30),
        'path' => '/',
        'httponly' => true,
        'samesite' => 'Lax'
    ]);
}
```

- Auto-login dari Cookie

```
// Jika session belum ada, cek Cookie Remember Me
if (!isset($_SESSION['user_id']) && isset($_COOKIE['remember_me'])) {
    list($userId, $token) = explode(':', $_COOKIE['remember_me'], 2);

    if ($userId && $token) {
        $stmt = $pdo->prepare("SELECT * FROM users WHERE id = :id");
        $stmt->execute(['id' => $userId]);
        $user = $stmt->fetch(PDO::FETCH_ASSOC);

        if ($user && $user['remember_token']) {
            if (hash_equals($user['remember_token'], hash('sha256', $token))) {
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['nama'] = $user['nama'];
                $_SESSION['role'] = $user['role'];
            }
        }
    }
}
```

- Hapus Cookie & Token

```
// Hapus token di database jika pengguna terautentikasi
if (isset($_SESSION['user_id'])) {
    $stmt = $pdo->prepare("UPDATE users SET remember_token = NULL WHERE id = :id");
    $stmt->execute(['id' => $_SESSION['user_id']]);
}

// Hapus cookie
if (isset($_COOKIE['remember_me'])) {
    setcookie('remember_me', '', [
        'expires' => time() - 3600,
        'path' => '/'
    ]);
}
```

- Pembahasan Risiko Keamanan (Security Discussion)

Penyimpanan Data

- a. Sesi Biasa (\$_SESSION)
Disimpan aman di *server-side*, browser hanya menyimpan Session ID sementara.
- b. Cookie "Remember Me" (setcookie)
Disimpan di *client-side* (browser pengguna) dalam jangka waktu lama.

Vulnerabilitas utama

- a. Sesi Biasa (\$_SESSION)
Hilang begitu browser ditutup atau masa berlaku sesi habis.
- b. Cookie "Remember Me" (setcookie)
Cookie Hijacking / Theft: Jika perangkat/browser diakses orang lain, token cookie dapat dicuri.

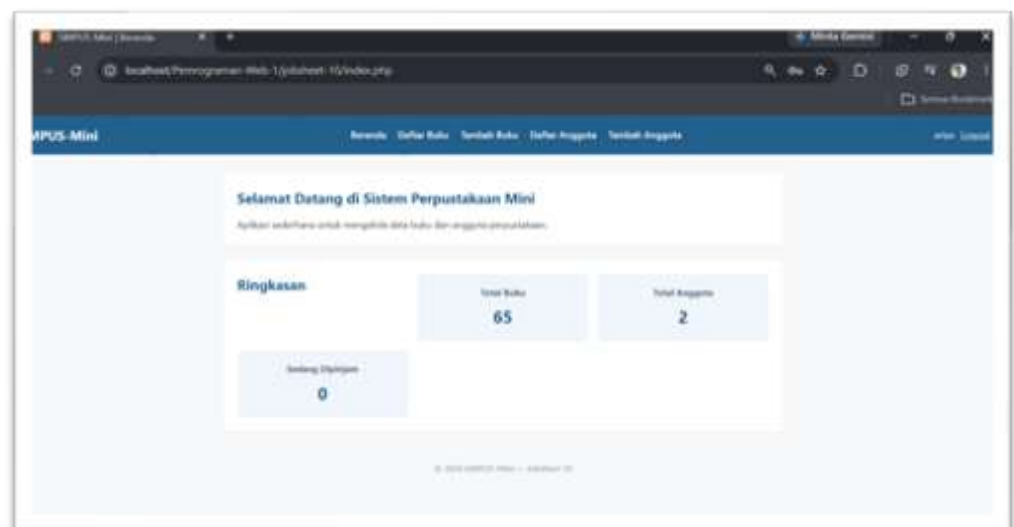
Mitigasi Keamanan

- a. Sesi Biasa (\$_SESSION)
Penanganan standar PHP Session.
- b. Cookie "Remember Me" (setcookie)
 1. Gunakan opsi HttpOnly agar Cookie tidak bisa diakses lewat script JavaScript (mencegah XSS).
 2. Jangan pernah menyimpan password atau data sensitif di Cookieplaintext.
 3. Simpan Hash token di DB, bukan token aslinya.
 4. Lakukan token invalidation saat pengguna menekan tombol Logout.

- Uji coba Fitur "Ingat Saya" (Remember Me)
 - a. Langkah 1 (Pengujian Login Persisten):
Buka auth/login.php.
Masukkan Username & Password, centang opsi "Ingat Saya", lalu klik Masuk.

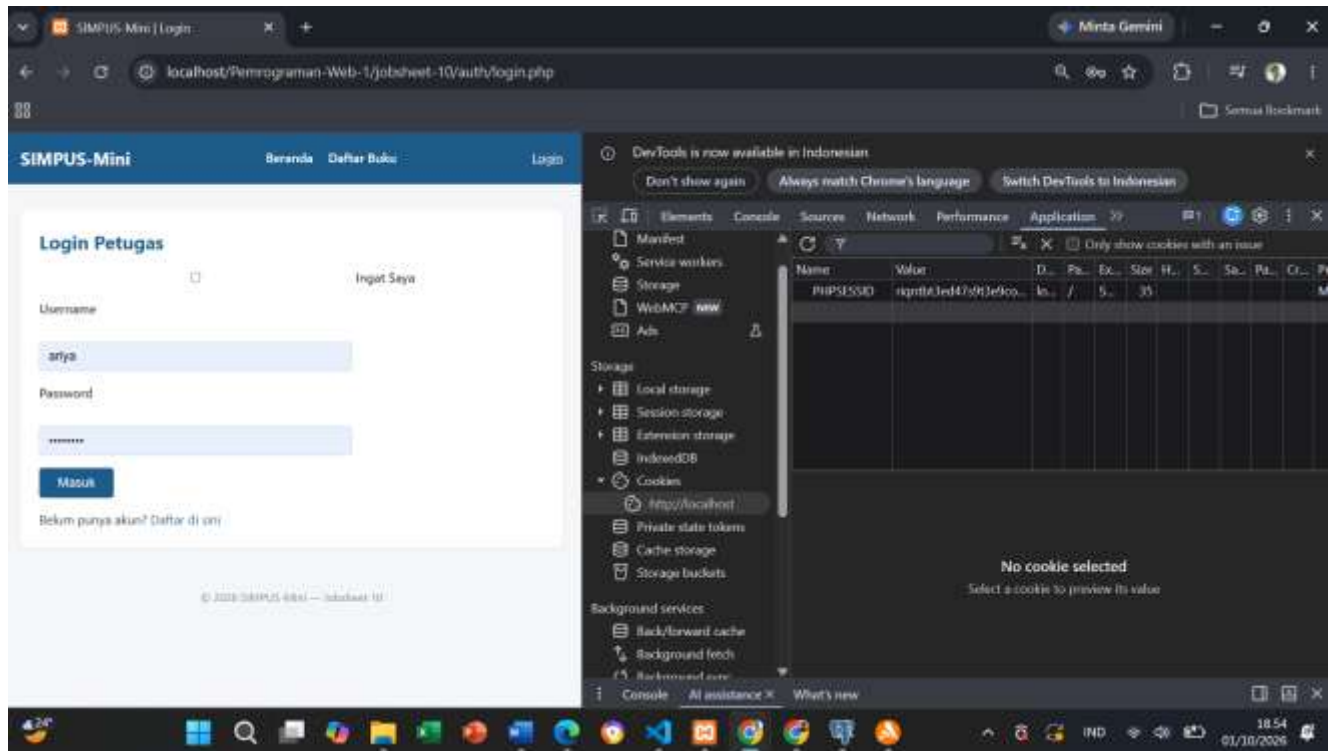
Pastikan masuk ke Beranda (index.php).
Tutup seluruh browser (bukan sekadar menutup tab).

- b. Langkah 2 (Pengujian Autologin):
Buka kembali browser dan langsung akses alamat `http://localhost/.../index.php` atau `anggota/list.php` tanpa melalui proses login manual.



Tetap dalam keadaan ter-login (Session otomatis kembali aktif dari validasi Cookie).

- c. Langkah 3 (Pengujian Logout):
Klik tombol Logout.
Buka kembali halaman index.php



Sistem berhasil menghapus cookie dan mengarahkan kembali pengguna ke halaman login.php

3. Batasi Percobaan Login yang Gagal (Mencegah Brute-Force)

- Menampilkan pesan peringatan / pemblokiran

```
// Cek status pemblokiran akibat brute-force
$isBlocked = false;
$timeRemaining = 0;
if (isset($_SESSION['lockout_time'])) {
    if (time() < $_SESSION['lockout_time']) {
        $isBlocked = true;
        $timeRemaining = $_SESSION['lockout_time'] - time();
    } else {
        // Durasi pemblokiran telah selesai, reset penghitung
        unset($_SESSION['lockout_time']);
        unset($_SESSION['login_attempts']);
    }
}
```

```
←!— Tampilkan Notifikasi Flash Jika Ada →
<?php if (isset($_SESSION['flash'])): ?>
    <div style="color: red; padding: 10px; border: 1px solid red; margin-bottom: 10px;">
        <?= htmlspecialchars($_SESSION['flash']['pesan']); ?>
    </div>
    <?php unset($_SESSION['flash']); ?>
<?php endif; ?>
```

```
←!— Tampilkan Peringatan Jika Akun Terkunci →
<?php if ($isBlocked): ?>
    <div style="color: darkred; background-color: #fee; padding: 10px; border: 1px solid red;">
        Terlalu banyak percobaan login yang gagal! Silakan coba lagi dalam <?= $timeRemaining; ?> detik.
    </div>
<?php else: ?>
```

- Menghitung Kegagalan Login

```
// Cek apakah user sedang dalam masa pemblokiran
if (isset($_SESSION['lockout_time']) && time() < $_SESSION['lockout_time']) {
    $_SESSION['flash'] = [
        'type' => 'error',
        'pesan' => 'Login ditolak. Anda masih dalam masa tunggu akibat terlalu banyak kesalahan.'
    ];
    header('Location: login.php');
    exit;
}
```

```
// Inisialisasi penghitung jika belum ada
if (!isset($_SESSION['login_attempts'])) {
    $_SESSION['login_attempts'] = [];
}

if (!isset($_SESSION['login_attempts'][$username])) {
    $_SESSION['login_attempts'][$username] = 0;
}
```

```

// Login berhasil: Reset penghitung kegagalan
unset($_SESSION['login_attempts'][$username]);
unset($_SESSION['lockout_time']);

$_SESSION['user_id'] = $user['id'];
$_SESSION['nama'] = $user['nama'];
$_SESSION['role'] = $user['role'];

// Jika gagal login: Tambah jumlah percobaan
$_SESSION['login_attempts'][$username]++;
$maxAttempts = 3;
$lockoutDuration = 300; // 5 menit

```

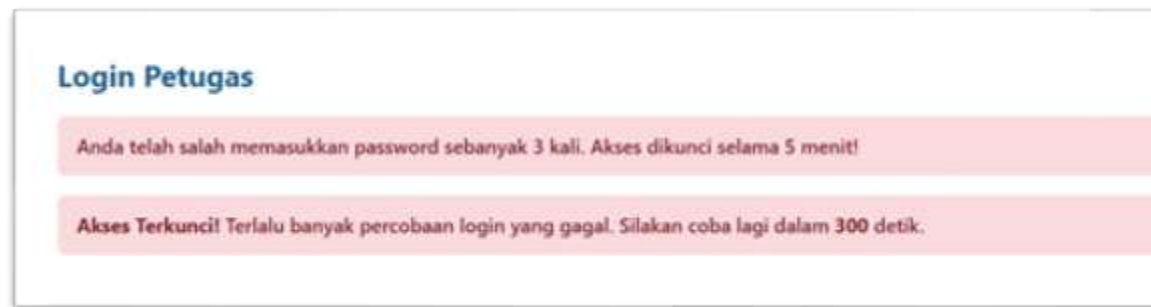
Pembatasan Percobaan Login yang Gagal (Mencegah Brute-Force): Telah diimplementasikan fitur pembatasan percobaan login berbasis Session untuk mengantisipasi serangan brute-force pada halaman autentikasi. Sistem menghitung akumulasi kegagalan autentikasi berdasarkan username melalui `$_SESSION['login_attempts']`. Apabila pengguna mengalami kegagalan login sebanyak 3 kali berturut-turut, sistem akan menetapkan durasi pemblokiran (lockout time) selama 5 menit (300 detik). Selama masa kunci aktif, formulir login akan disembunyikan dan sistem menampilkan notifikasi penolakan akses beserta durasi sisa waktu tunggu.

- Uji coba Pembatasan Brute-Force
 - a. Buka halaman `auth/login.php`.
 - b. Masukkan username acak (misal: petugas) dan password yang salah.
 - c. Klik tombol Masuk sebanyak 1 kali.

The screenshot shows a web form titled "Login Petugas". At the top, there is a red error message: "Username atau password salah. Sisa percobaan: 2". Below this, the form has two input fields: "Username" with the value "ariya" and "Password" which is masked with dots. There is a "Masuk" button and a link "Ingat Saya". At the bottom, there is a link "Belum punya akun? Daftar di sini".

Muncul notifikasi "Username atau password salah. Sisa percobaan: 2".

- d. Ulangi memasukkan password yang salah hingga total 3 kali percobaan.



- e. Setelah percobaan ke-3, formulir login terkunci/hilang dan muncul notifikasi "Terlalu banyak percobaan login yang gagal! Silakan coba lagi dalam 300 detik."

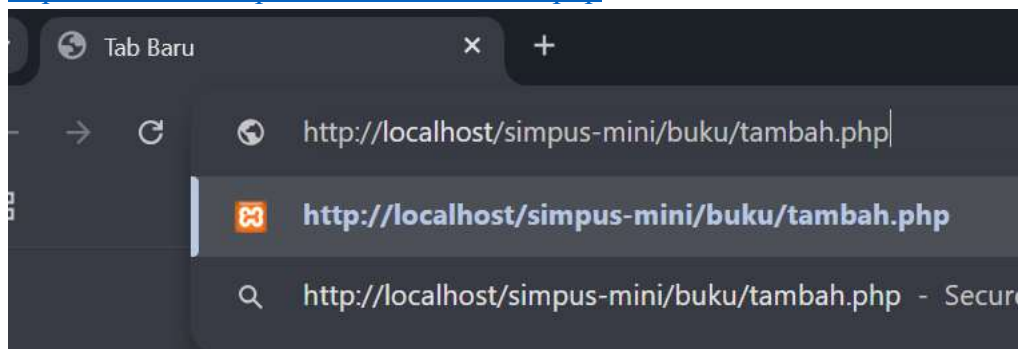
4. Pembuktian Uji Coba Mematikan PostgreSQL

Uji Coba Mematikan Layanan PostgreSQL (Hirarki Eksekusi Kode): Telah dilakukan pengujian terhadap urutan eksekusi sistem dengan mematikan (stop) layanan (service) PostgreSQL pada lingkungan pengujian. Saat halaman terlindungi seperti /buku/tambah.php diakses tanpa sesi login aktif, pengguna secara otomatis langsung dialihkan (redirect) ke halaman auth/login.php tanpa memicu Fatal Error atau exception koneksi database. Hal ini mengonfirmasi kebenaran konsep pada Bab 4 §4.6, di mana pustaka auth.php dipanggil lebih awal daripada pustaka koneksi.php, sehingga guard otentikasi bekerja secara efisien mencegah beban koneksi database yang tidak perlu.

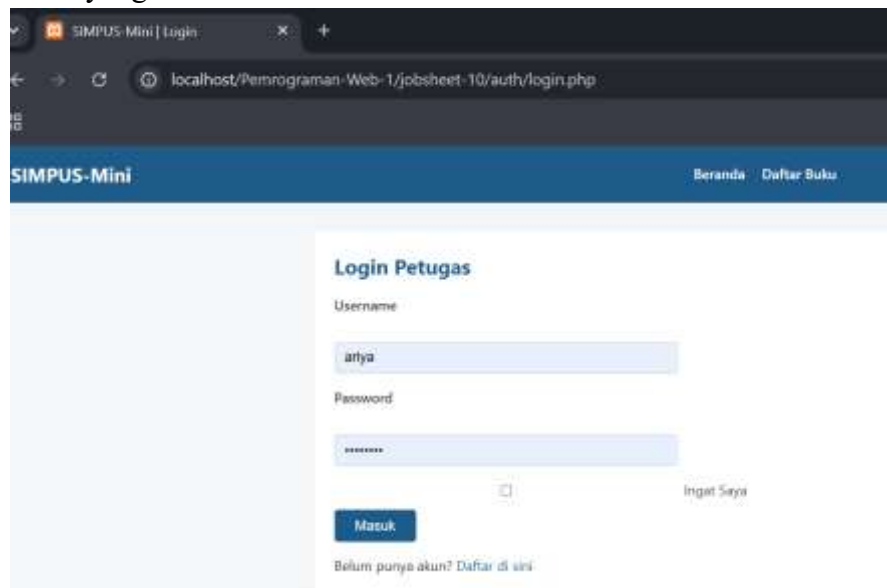
Uji coba

- Mematikan PostgreSQL:
Mengganti nilai variabel password menjadi salah:

```
$pass = "password salah";
```
- Pengujian Akses:
Pastikan Anda dalam keadaan Logged Out (atau gunakan mode Incognito/Private Window).
Langsung ketik URL halaman terlindungi di browser, misalnya:
<http://localhost/simpus-mini/buku/tambah.php>



- Hasil yang Diamati:



Browser TIDAK menampilkan pesan error seperti PDOException: SQLSTATE[08006] Connection refused.

Pengguna secara mulus langsung diarahkan ke halaman <http://localhost/simpus-mini/auth/login.php>. Ini membuktikan proteksi auth.php dipanggil sebelum file koneksi database mengeksekusi query.

JOBSHEET 11

1. Penambahan Proteksi CSRF pada Form Pencarian (buku/list.php)

Diskusi Risiko Form GET vs. Form POST pada Attack Vector CSRF

- Form GET (Read-Only Data): Method GET dirancang untuk operasi yang sifatnya idempotent dan safe (membaca/menampilkan data tanpa mengubah status atau isi database). Jika penyerang memicu serangan CSRF pada form GET pencarian, aplikasi hanya akan menampilkan hasil pencarian tanpa merusak data. Selain itu, menyertakan token CSRF pada URL GET dapat menyebabkan token bocor melalui header Referer, log server, atau riwayat peramban web (browser history).
- Form POST (State-Changing Operations): Method POST digunakan untuk mengubah state server atau isi database (seperti Insert, Update, Delete). Jika tidak dilindungi token CSRF, penyerang bisa menjebak korban untuk mengeksekusi aksi merusak (misalnya menghapus data) tanpa sepengetahuan pengguna.
- Tambahkan fungsi csrf_field() di dalam tag <form method="get" action="list.php">:

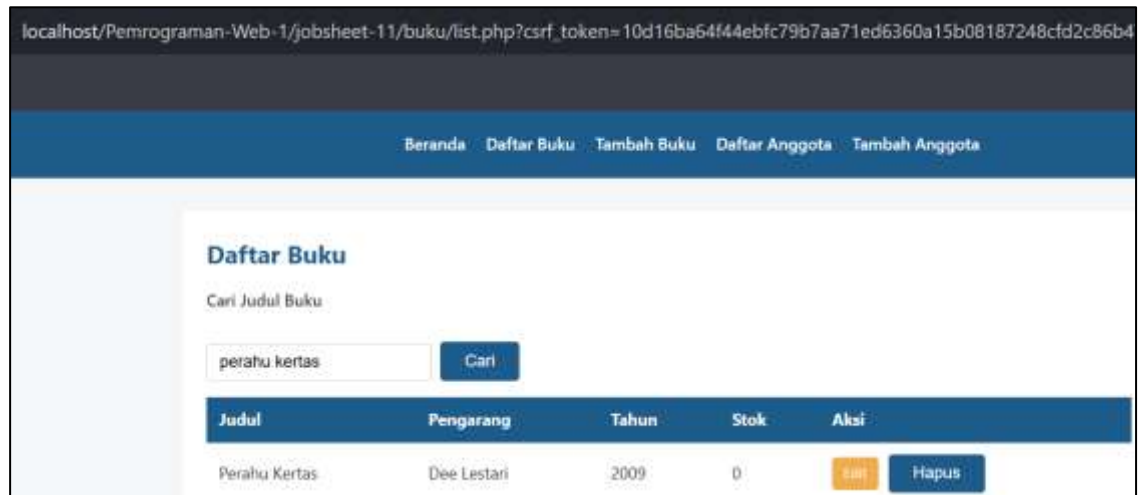
```
<div class="search-box">
  <form method="get" action="list.php">
    <?php echo csrf_field(); ?>
    <span>
      <label for="search-input">Cari Judul Buku</label><br>
      <input type="text" id="search-input" name="q" value="<?php echo e($keyword); ?>"
        placeholder="Ketik judul buku ... ">
    </span>
    <button type="submit">Cari</button>
  </form>
</div>
```

- Validasi Token di buku/list.php
Tambahkan verifikasi token di bagian awal file buku/list.php (setelah impor csrf.php melalui header.php):

```
// Verifikasi CSRF untuk method GET jika ada pencarian
if (isset($_GET['q'])) {
    $token = $_GET['csrf_token'] ?? '';
    if ($token === '' || !hash_equals($_SESSION['csrf_token'] ?? '', $token)) {
        http_response_code(403);
        die('Permintaan ditolak: token CSRF tidak valid atau kedaluwarsa.');
```

- Uji Coba (Pengujian Proteksi CSRF Pencarian)

Langkah: Buka halaman buku/list.php, ketik kata kunci pada kolom pencarian perahu kertas, lalu klik tombol Cari



URL otomatis menyertakan csrf_token yang valid. Sistem berhasil memverifikasi token dan menampilkan hasil pencarian dengan normal.

Uji Coba 2: Pengujian Serangan CSRF (Token Salah / Tanpa Token)

Langkah: Buka URL secara langsung tanpa token atau dengan mengganti nilai token pada URL contoh =

http://localhost/Pemrograman-Web-1/jobsheet-

11/buku/list.php?csrf_token=salah123&q=perahu+kertas



sistem langsung memblokir akses dan memunculkan pesan "Permintaan ditolak: token CSRF tidak valid atau kedaluwarsa."

2. Penambahan Baris Audit pada security-checklist.md

menambahkan item audit terkait kebocoran pesan error mentah PHP / database ke pengguna:

5	Session Fixation	auth/proses_login.php	Session ID tidak diperbarui setelah login	session_regenerate_id(true) dipanggil tepat setelah password_verify() berhasil.
6	Information Disclosure (Pesan Error DB)	includes/koneksi.php	Pesan error koneksi PDO mentah (\$e->getMessage()) ditampilkan langsung ke pengguna lewat die() , berisiko membocorkan struktur database/server.	Detail error ditangkap dan dicatat ke log server (error_log()), sedangkan pengguna hanya melihat pesan generik yang aman seperti "Koneksi database gagal.*".

3. Menerapkan fungsi e() pada tempat yang belum dibungkus:

Berdasarkan audit kode pada aplikasi, penerapan fungsi e() untuk mencegah serangan XSS (Cross-Site Scripting) telah ditambahkan pada beberapa berkas yang sebelumnya mencetak data mentah. Perbaikan dilakukan pada:

1. includes/header.php: Membungkus \$_SESSION['nama'] dengan e() saat menampilkan nama pengguna yang sedang login

```
<div class="auth-status">
  <?php if ($sudahLogin): ?>
    <span><?php echo e($_SESSION['nama']); ?></span>
    <a href="<?php echo $base; ?>auth/logout.php">Logout</a>
  <?php else: ?>
```

2. buku/edit.php: Membungkus variabel \$value dan \$label pada pilihan <option> kategori buku.

```
<select id="kategori" name="kategori">
  <?php foreach (['fiksi' => 'Fiksi', 'non-fiksi' => 'Non-Fiksi', 'referensi' => 'Referensi'] as
  $value => $label): ?>
    <option value="<?php echo e($value); ?>" <?php echo $buku['kategori'] == $value ? 'selected'
    : ''; ?><?php echo e($label); ?></option>
  <?php endforeach; ?>
</select>
```

3. Pesan Flash di seluruh modul (buku/, anggota/, auth/): Membungkus \$flash['type'] dan \$flash['pesan'] pada berkas tambah.php, edit.php, list.php, login.php, dan register.php untuk memastikan pesan error atau sukses aman dari injeksi skrip.

- Buku list

```
<h2>Daftar Buku</h2>

<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- Buku edit

```
<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- Buku tambah

```
<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- Edit anggota

```
<h2>Edit Anggota</h2>

<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- List anggota

```
<h2>Daftar Anggota</h2>

<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- Tambah anggota

```
<h2>Tambah Anggota</h2>

<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- Login

```
<h2>Login Petugas</h2>

<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

- Register

```
<h2>Registrasi Petugas</h2>

<?php if ($flash): ?>
  <p class="flash flash-<?php echo e($flash['type']); ?>"><?php echo e($flash['pesan']); ?></p>
<?php endif; ?>
```

4. Uji Coba 1: XSS via Nama User (header.php)

Buka halaman auth/register.php.

Daftar akun baru dengan nama: `<script>alert('XSS Header')</script>`.

Login menggunakan akun tersebut.

Registrasi Petugas

Nama

Username

Password



Teks skrip akan muncul sebagai tulisan biasa di pojok kanan atas (header) dan tidak memicu pop-up alert browser.

4. Penjelasan Content-Security-Policy (CSP) sebagai Lapis Pertahanan XSS

Content-Security-Policy (CSP) adalah header HTTP respons yang memungkinkan pengembang membatasi sumber daya (seperti JavaScript, CSS, gambar) yang boleh dimuat dan dieksekusi oleh browser.

CSP bekerja sebagai lapisan pertahanan tambahan (defense in depth) terhadap serangan XSS dengan cara:

- Memblokir Inline Script secara Default: Aturan CSP yang ketat (misalnya `script-src 'self'`) secara otomatis menolak eksekusi inline JavaScript seperti `<script>...</script>` atau event handler HTML seperti `onclick=""`.
- Membatasi Sumber Eksekusi: Browser hanya diperbolehkan menjalankan file JavaScript yang berasal dari domain yang sudah terverifikasi dan terdaftar dalam kebijakan CSP.

Skenario Perlindungan:

Seandainya ada satu tempat di kode PHP yang secara tidak sengaja lupa dibungkus fungsi `e()` sehingga payload XSS lolos ke dalam HTML, browser yang membaca header CSP akan menolak mengeksekusi skrip jahat tersebut. Dengan demikian, serangan XSS tetap gagal dan aplikasi tetap aman.